

Proposal: USD Forex Data API Service for Power BI

1. Objective

We propose building a USD forex data service that uses Dukascopy as the upstream source, stores historical data in our own system, updates the dataset once per day, and exposes the data through an API for both a website and Power BI.

The purpose of this project is to create a stable and reusable data layer for analysis. Instead of letting Power BI depend directly on Dukascopy's website, we will use Dukascopy only as the raw source and build our own backend service as the actual data product.

This design supports three goals:

- centralized historical data storage
- daily automated updates
- direct API-based integration with Power BI

2. Chosen Solution

We will use a historical full cache plus daily incremental update architecture.

The workflow will be:

Dukascopy -> ingestion job -> local storage -> API -> website + Power BI

This means:

- first, we collect and store historical USD-related forex data
- then, we update it once per day
- finally, we serve the data through our own API

This approach avoids repeated manual downloads and creates a system that is easier to maintain and extend.

3. Scope of Currency Pairs

The project will include all forex pairs involving USD from the Dukascopy list.

Examples include:

- AUD/USD
- EUR/USD
- GBP/USD
- NZD/USD

- USD/CAD
- USD/CHF
- USD/CNH
- USD/CZK
- USD/DKK
- USD/HKD
- USD/HUF
- USD/ILS
- USD/JPY
- USD/MXN
- USD/NOK
- USD/PLN
- USD/RON
- USD/SEK
- USD/SGD
- USD/THB
- USD/TRY
- USD/ZAR

The system should treat this as the official USD pair universe for the project.

4. Pair Metadata Fields

For the currency pair reference list, we will use only the fields shown in the Dukascopy instrument view:

- pair
- pair_name

Examples:

- AUD/USD | Australian Dollar vs US Dollar
- EUR/USD | Euro vs US Dollar
- USD/JPY | US Dollar vs Yen

This keeps the pair list aligned with the source website and avoids introducing unnecessary metadata at this stage.

5. System Components

A. Pair Catalog

A reference list of all USD-related forex pairs from Dukascopy.

Purpose:

- define the supported symbols
- provide a clean source for dropdowns, filtering, and API validation
- avoid hardcoding pair values in multiple places

Fields:

- pair
- pair_name

B. Historical Backfill Process

A one-time or on-demand process that retrieves historical data for all supported USD pairs over the selected lookback period.

Purpose:

- build the initial full dataset
- avoid repeated full downloads later
- make Power BI access much faster

C. Daily Incremental Update Process

A scheduled job that runs once per day and fetches only newly available data from Dukascopy.

Purpose:

- keep the local dataset current
- reduce repeated processing
- support future dashboard refreshes

D. Local Storage

A structured data store that keeps:

- the supported pair list
- historical price data
- update logs

E. API Layer

A backend API that allows the website and Power BI to request data using a stable query interface.

F. Website

A lightweight front-end where users can:

- choose number of years
- choose all USD pairs or selected pairs

- preview available pair names
- call the same backend API that Power BI uses

6. Functional Requirements

The system should support the following functions:

Pair Management

- maintain a supported USD pair catalog
- expose the pair list through API
- use only `pair` and `pair_name` for pair metadata

Historical Data Loading

- retrieve historical data from Dukascopy
- load all supported USD pairs
- support configurable lookback windows such as 1, 3, or 5 years

Daily Updates

- run once per day
- retrieve only new missing records
- append new records into local storage

API Access

- allow Power BI to request all USD pairs
- allow filtering by specific pairs
- allow filtering by years or custom date range
- provide a stable response schema

Website Support

- display the pair list
- allow parameter selection
- call the API for download or preview

7. Recommended API Contract

To make the system ready for implementation, the API should be designed around the following endpoints.

Pair Catalog Endpoint

Purpose:

- return the supported USD pair list

Example:

- `GET /api/pairs`

Response fields for each item:

- pair
- pair_name

All USD Pairs Data Endpoint

Purpose:

- return price data for the full USD universe

Example:

- GET /api/forex/usd-pairs?years=3
- GET /api/forex/usd-pairs?start_date=2023-01-01&end_date=2026-03-25

Selected Pairs Data Endpoint

Purpose:

- return price data for a subset of pairs

Example:

- GET /api/forex/pairs?symbols=AUD/USD,EUR/USD,USD/JPY&years=1

Health or Status Endpoint

Purpose:

- show API availability and latest update information

Example:

- GET /api/status

This endpoint structure is simple enough for development and clean enough for Power BI integration.

8. Data Design for Implementation

To make coding easier, the system should separate the data model into three logical parts.

A. Pair Reference Data

Used for supported instruments and UI dropdowns.

Fields:

- pair
- pair_name

B. Historical Price Data

Used for analytics and Power BI.

This dataset should store the actual market data retrieved from Dukascopy. Its exact fields can follow the raw historical dataset returned by Dukascopy, but the pair reference itself should still link back to the two-field pair catalog above.

C. Update Log

Used for pipeline control.

Recommended tracked values:

- last successful run date
- pair processed
- record count loaded
- update status

This separation makes the codebase cleaner and easier to test.

9. Update Logic

The update logic should work in two phases.

Phase 1: Backfill

- read the USD pair catalog
- fetch the historical range for each pair
- store the result locally

Phase 2: Daily Refresh

- check the last stored date for each pair
- fetch only missing new data from Dukascopy
- append new records
- write update status to the log

This gives the project a predictable and efficient workflow.

10. Why This Is the Best Fit

This proposal is a strong fit for the project because it balances simplicity and scalability.

It is better than connecting Power BI directly to Dukascopy because:

- the source website is not designed as a stable BI backend
- direct dependence on the UI is fragile
- refresh behavior is harder to control

It is better than static file download only because:

- it supports daily automated updates
- it creates a reusable API
- it allows both website access and Power BI access from one backend

It is also more realistic for a capstone project than building a true real-time streaming system.

11. Build Order for the Next Step

To make this ready for coding, the development sequence should be:

1. Define the official USD pair catalog using only:
 - pair
 - pair_name
2. Build the historical ingestion process for all supported pairs
3. Design local storage for:
 - pair reference data
 - historical price data
 - update logs
4. Build the daily incremental update job
5. Build API endpoints:
 - /api/pairs
 - /api/forex/usd-pairs
 - /api/forex/pairs
 - /api/status
6. Build a simple website UI that consumes the same API
7. Connect Power BI to the API

This order keeps the project modular and easy to divide across team members.

12. Final Recommendation

We recommend building an API-based USD forex data service with:

- Dukascopy as the upstream source
- a local historical cache
- daily incremental updates
- a pair catalog defined only by pair and pair_name
- Power BI connected to our API rather than directly to Dukascopy

This design gives us a stable, maintainable, and implementation-ready foundation for the next stage of the project.